

Practical No. 2

Aim:Write a JAX-WS web service to convert Fahrenheit to Celsius and vice versa. Define a JSP that consumes the web service.

Steps:

1. Create a webApplication

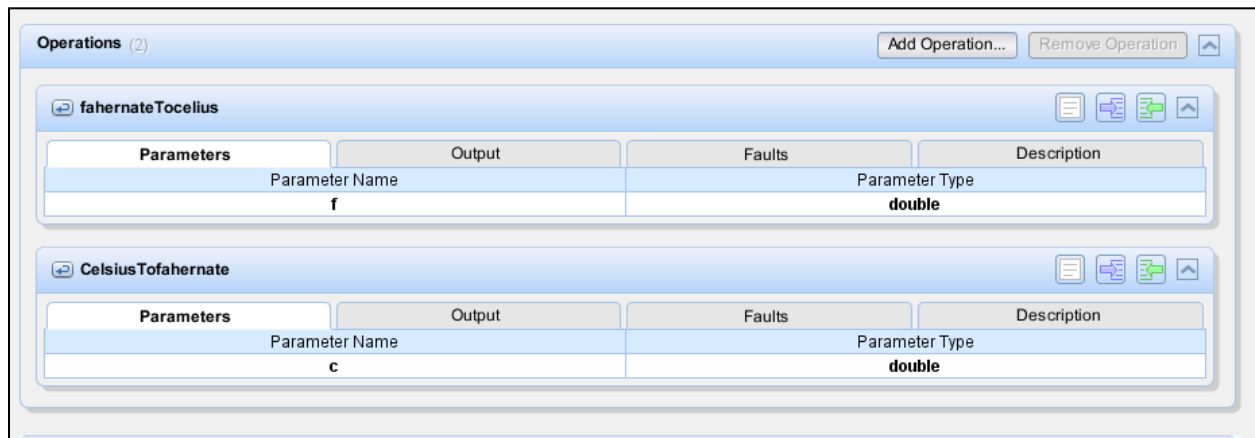
2. Create a web service (abc.com)

```
package com.myApp;
import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;
* @author Administrator
*/
@WebService(serviceName = "TemperatureConverterws")
public class TemperatureConverters {

    /**
     * Web service operation
     */
    @WebMethod(operationName = "fahrenheitToCelsius")
    public Double fahrenheitToCelsius(@WebParam(name = "f") double f) {
        //TODO write your implementation code here:
        return (f-32)*5.0/9.0;
    }

    /**
     * Web service operation
     */
    @WebMethod(operationName = "CelsiusToFahrenheit")
    public Double CelsiusToFahrenheit(@WebParam(name = "c") double c) {
        //TODO write your implementation code here:
        return (c*9.0/5.0)+32;
    }
}
```

3. Go to design for creating new methods and create new methods



4. After creating methods clean and build the project then deploy.

5. Then Click on test web services.

6. Click on WSDL File and Copy URL.

TemperatureConverterws Web Service Tester

This form will allow you to test your web service implementation ([WSDL File](#))

To invoke an operation, fill the method parameter(s) input boxes and click on the button labeled with the method name.

Methods :

public abstract java.lang.Double com.myapp.TemperatureConverterws.celsiusToFahrenheit(double)

celsiusToFahrenheit (80)

public abstract java.lang.Double com.myapp.TemperatureConverterws.fahrenheitToCelsius(double)

fahrenheitToCelsius (32)

celsiusToFahrenheit Method invocati

Method parameter(s)

Type	Value
double	32

Method returned

java.lang.Double : "89.6"

fahernateTocelius Method invocation					
Method parameter(s)					
<table><tr><th>Type</th><th>Value</th></tr><tr><td>double</td><td>88</td></tr></table>	Type	Value	double	88	
Type	Value				
double	88				
Method returned					
java.lang.Double : "26.666666666666668"					

7. Create a java Application.

8. Right Click on Java Application go to new web service client.

9. Paste the URL on WSDL URL option.

\

Index.html

<!DOCTYPE html>

<!--

To change this license header, choose License Headers in Project Properties.

To change this template file, choose Tools | Templates

and open the template in the editor.

-->

<html>

<head>

<title>Saksh Parande 9078</title>

</head>

<body>

<h2>Temperature Converter</h2>

<form>

Enter Temperature: <input type="text" name="temp">

<input type="submit" formaction="ftoc.jsp" value="Celsius">

<input type="submit" formaction="ctof.jsp" value="Fahrenheit">

</form>

</body>

</html>

10. Just drag and drop methods in jsp pages

Ftoc.jsp

try {

com.myapp.TemperatureConverterws_Service service = new
com.myapp.TemperatureConverterws_Service();

com.myapp.TemperatureConverterws port = service.getTemperatureConverterwsPort();

// TODO initialize WS operation arguments here

```

double f = Double.parseDouble(request.getParameter("temp"));
// TODO process result here
java.lang.Double result = port.fahernateTocelsius(f);
out.println("Result = "+result);
} catch (Exception ex) {
    // TODO handle custom exceptions here
}
%>
<%-- end web service invocation --%><hr/>

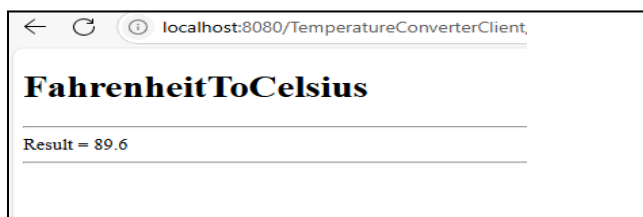
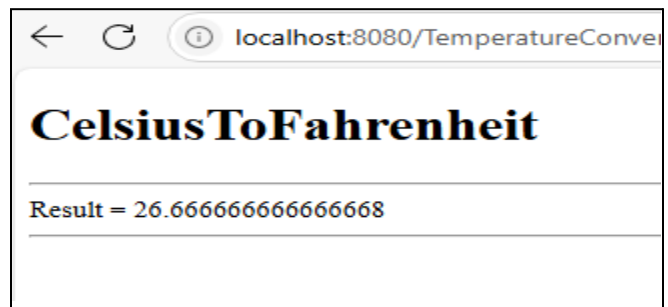
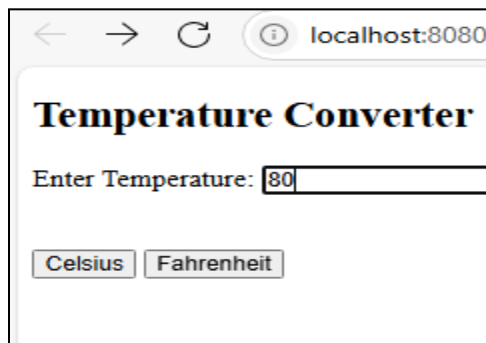
```

Ctof.jsp

```

try {
    com.myapp.TemperatureConverterws_Service service = new
com.myapp.TemperatureConverterws_Service();
    com.myapp.TemperatureConverterws port = service.getTemperatureConverterwsPort();
    // TODO initialize WS operation arguments here
    double c = Double.parseDouble(request.getParameter("temp"));
    // TODO process result here
    java.lang.Double result = port.celsiusToFahernate(c);
    out.println("Result = "+result);
} catch (Exception ex) {
    // TODO handle custom exceptions here
}
%>
<%-- end web service invocation --%><hr/>

```



Practical no :3

Create a web service as Weather Information. Develop a client that consumes web services developed in different platforms. (Java to .NET and .NET to Java)

weather_app.java

```
package weather_app;
import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;

/**
 *
 * @author Administrator
 */
@WebService(serviceName = "java")
public class java {

    /**
     * This is a sample web service operation
     */
    @WebMethod(operationName = "weatherforcaster")
    public String weatherforcaster(@WebParam(name = "city") String city) {
        if(city.equals("Navi- Mumbai")){
            return "Heavy Rainfall";
        }
        else if(city.equals("Mumbai")){
            return "No Rainfall";
        }
        else{
            return "No weather information";
        }
    }
}
```

weatherServices Web Service Tester

This form will allow you to test your web service implementation ([WSDL File](#))

To invoke an operation, fill the method parameter(s) input boxes and click on the button labeled with the method name.

Methods :

```
public abstract java.lang.String abc.com.WeatherServices.weatherInfo(java.lang.String)
```

weatherInfo (navi-Mumbai)

weatherInfo Method invocation

Method parameter(s)

Type	Value
java.lang.String	navi-Mumbai

Method returned

java.lang.String : "Heavy Rainfall"

weatherInfo Method invocation

Method parameter(s)

Type	Value
java.lang.String	Mumbai

Method returned

java.lang.String : "no rainfall"

Visual Studio Step

Create a new project

Recent project templates

- ASP.NET Web Application (.NET Framework) C#
- Windows Forms App (.NET Framework) C#
- ASP.NET Core Web App C#

Asp.net web

C# Windows Web

Core MVC Views and Controllers.

C# Linux macOS Windows Cloud Service Web

WebAPI

ASP.NET Core Web App (Model-View-Controller)

A project template for creating an ASP.NET Core application with example ASP.NET Core MVC Views and Controllers. This template can also be used for RESTful HTTP services.

C# Linux macOS Windows Cloud Service Web

ASP.NET Web Application (.NET Framework)

Project templates for creating ASP.NET applications. You can create ASP.NET Web Forms, MVC, or Web API applications and add many other features in ASP.NET.

C# Windows Cloud Web

Configure your new project

ASP.NET Web Application (.NET Framework) C# Windows Cloud Web

Project name

WeatherClientApp

Location

C:\Users\Administrator\source\repos

Solution name

WeatherClientApp

Create a new ASP.NET Web Application

Empty

An empty project template for creating ASP.NET applications. This template does not have any content in it.

Web Forms

A project template for creating ASP.NET Web Forms applications. ASP.NET Web Forms lets you build dynamic websites using a familiar drag-and-drop, event-driven model. A design surface and hundreds of controls and components let you rapidly build sophisticated, powerful UI-driven sites with data access.

MVC

A project template for creating ASP.NET MVC applications. ASP.NET MVC allows you to build applications using the

Authentication

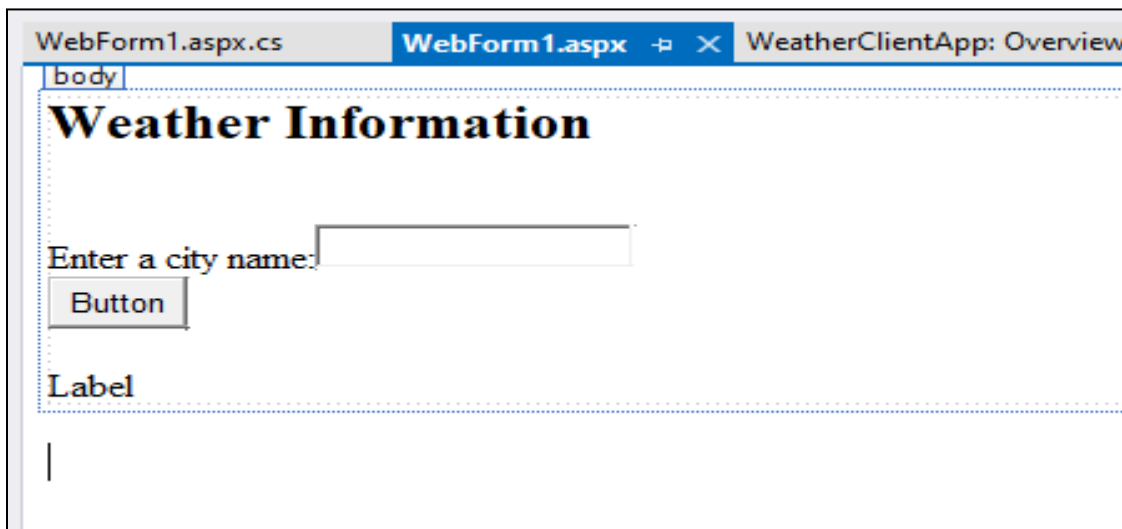
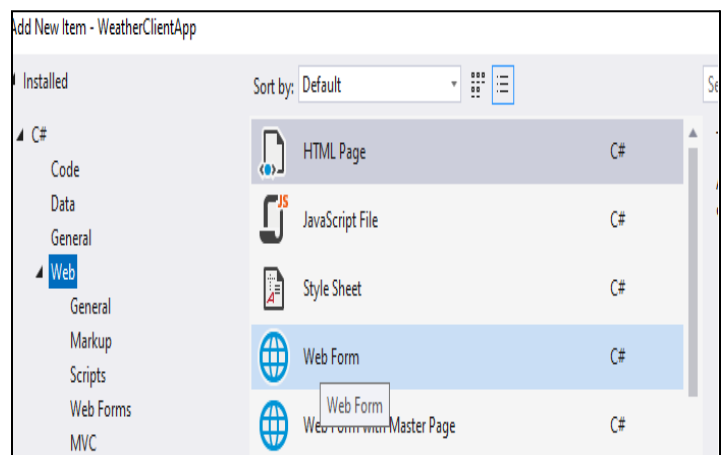
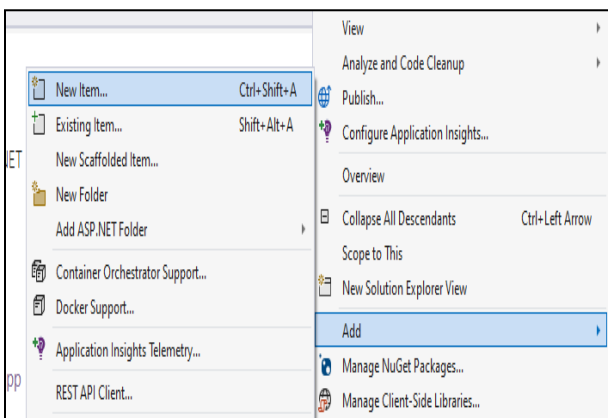
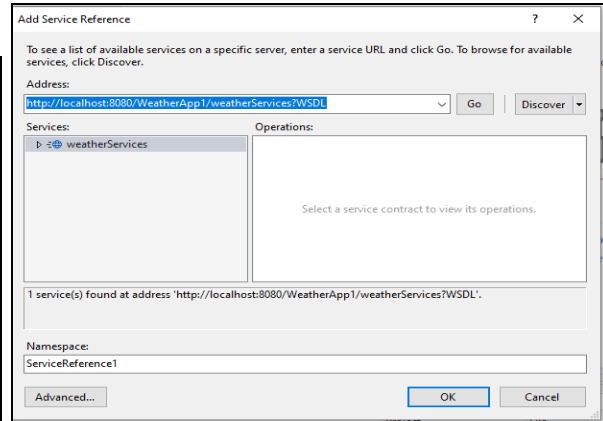
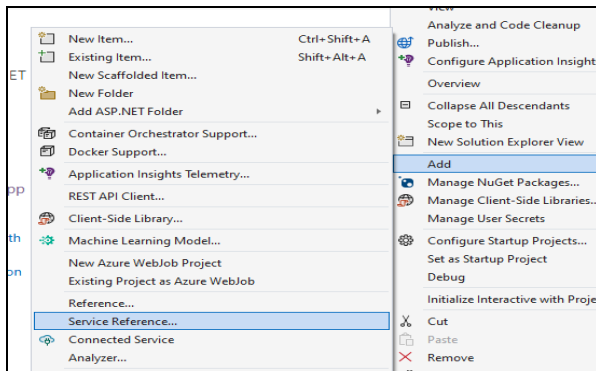
None

Add folder

- ☐ Web Form
- ☐ MVC
- ☐ Web API

Name:Sakshi Parande

Roll N0:9078(B)



WebForm1.aspx:-

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="WebForm1.aspx.cs"
Inherits="WeatherClientApp.WebForm1" %>
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title></title>
</head>
<body>
    <form id="form1" runat="server">
        <div>
            <h2>Weather Information</h2>
            <br />
            Enter a city name:<asp:TextBox ID="txtCity" runat="server"></asp:TextBox>
            <br />
            <asp:Button ID="Button1" runat="server" Text="Button"
OnClick="Button1_Click" />
            <br />
            <br />
            <asp:Label ID="Label1" runat="server" Text="Label"></asp:Label>
        </div>
    </form>
</body>
</html>
```

WebForm1.aspx.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
namespace WeatherClientApp
{
    public partial class WebForm1 : System.Web.UI.Page
    {
```



```
protected void Page_Load(object sender, EventArgs e)
{

}

protected void Button1_Click(object sender, EventArgs e)
{
    string city = txtCity.Text;
    ServiceReference1.weatherServicesClient s1
        = new ServiceReference1.weatherServicesClient();
    Label1.Text = s1.WeatherInfo(city);
}
}
```

A screenshot of a web browser window. The address bar shows 'localhost:44323/WebForm1.aspx'. The page title is 'Weather Information'. Below the title, there is a text input field with 'navi-Mumbai' and a 'Button' next to it. Below the button, the text 'Heavy Rainfall' is displayed.

A screenshot of a web browser window. The address bar shows 'localhost:44323/WebForm1.aspx'. The page title is 'Weather Information'. Below the title, there is a text input field with 'nagar' and a 'Button' next to it. Below the button, the text 'No Weather Information available' is displayed.

A screenshot of a web browser window. The address bar shows 'localhost:44323/WebForm1.aspx'. The page title is 'Weather Information'. Below the title, there is a text input field with 'Mumbai' and a 'Button' next to it. Below the button, the text 'no rainfall' is displayed.

Practical No:4

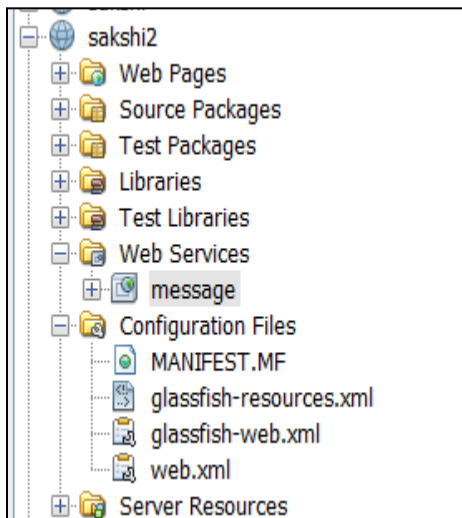
Aim: Write a program to implement the operation and can receive requests and will return a response in two ways. a) One - Way operation b) Request - Response

Code:

1)Message.javapackage abc.com;

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import javax.annotation.Resource;
import javax.jws.Oneway;
import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;
import javax.sql.DataSource;
/**
 *
 * @author Sakshi
 */
@WebService(serviceName = "message")
public class message {
    @Resource(name = "mydsn")
    private DataSource mydsn;
    /**
     * Web service operation
     */
    @WebMethod(operationName = "insertName")
    @Oneway
    public void insertName(@WebParam(name = "id") int id, @WebParam(name = "name")
String name) {
        try {
            Connection c = mydsn.getConnection();
            PreparedStatement ps = c.prepareStatement("insert into student(ID, name) values(?, ?)");
            ps.setInt(1, id);
            ps.setString(2, name);
            ps.executeUpdate();
        } catch (Exception e) {
            // Handle or log error
        }
    }
}
```

```
    }  
}  
/**  
 * Web service operation  
 */  
@WebMethod(operationName = "getName")  
public String getName(@WebParam(name = "ID") int ID) {  
    try {  
        Connection c = mydsn.getConnection();  
        PreparedStatement ps = c.prepareStatement("select * from student where id=?");  
        ps.setInt(1, ID);  
        ResultSet r = ps.executeQuery();  
        if (r.next())  
            return r.getString(2);  
        else  
            return "No name found";  
    } catch (Exception e) {  
        return "error";  
    }  
}  
}
```



message Web Service Tester

This form will allow you to test your web service implementation ([WSDL File](#))

To invoke an operation, fill the method parameter(s) input boxes and click on the button labeled with the method name.

Methods :

public abstract java.lang.String abc.com.Message.getName(int)

getName (1)

public abstract void abc.com.Message.insertName(int,java.lang.String)

insertName (6, shubham)

getName Method invocation

Method parameter(s)

Type	Value
int	1

Method returned

java.lang.String : "sakshi"

insertName Method invocation

Method parameter(s)

Type	Value
int	6
java.lang.String	shubham

WS00041: Service invocation threw an exception with message : n

Exceptions details : java.lang.NullPointerException



Index.html

```
<!DOCTYPE html>
```

```
<!--
```

To change this license header, choose License Headers in Project Properties.

To change this template file, choose Tools | Templates

and open the template in the editor.

```
-->
```

```
<html>
```

```
<head>
```

```
<title>TODO supply a title</title>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
</head>
```

```
<body>
```

```
<!-- One-way Operation Form -->
```

```
<form>
```

```
<h2>One-way Operation</h2><br>
```

```
<input type="text" name="txt1" placeholder="Enter ID"><br><br>
```

```
<input type="text" name="txt2" placeholder="Enter
```

```
Firstname"><br><br>
```

```
<input type="submit" formaction="insertData.jsp" value="Insert
```

```
Record"><br>
```

```
</form>
```

```
<h1>-----</h1>
```

```
<!-- Request-Response Operation Form -->
```

```
<form>
```

```
<h2>Request-Response operation</h2><br>
```

```
<input type="text" name="txt3" placeholder="Enter ID"><br><br>
```

```
<input type="submit" formaction="getData.jsp" value="Get
```

```
Firstname"><br><br>
```

```
</form> </body></html>
```

insertData.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>JSP Page</title>
  </head>
  <body>
    <h1>Hello World!</h1>
    <%-- start web service invocation --%><hr/>
    <%
      int fid = Integer.parseInt(request.getParameter("txt1"));
      String fname = request.getParameter("txt2");
      try {
        com.dd.Message_Service service = new com.dd.Message_Service();
        com.dd.Message port = service.getMessagePort();
        // TODO initialize WS operation arguments here
        int id = fid;
        java.lang.String name = fname;
        port.insertName(id, name);
      } catch (Exception ex) {
        }
    %>
    <%-- end web service invocation --%><hr/>
  </body>
</html>
```

getData.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>JSP Page</title>
  </head>
  <body>
    <h1>Hello World!</h1>
    <%-- start web service invocation --%><hr/>
    <%
```

```
int fid = Integer.parseInt(request.getParameter("txt3"));
try {
    com.dd.Message_Service service = new com.dd.Message_Service();
    com.dd.Message port = service.getMessagePort();
    // TODO initialize WS operation arguments here
    int id = fid;
    // TODO process result here
    java.lang.String result = port.getName(id);
    out.println("name = "+result);
} catch (Exception ex) {
}
%>
<%-- end web service invocation --%><hr/>
</body></html>
```

One-way Operation

Hello World!

Request-Response operation

Hello World!

name = sakshi

Name:Sakshi Parande

Roll N0:9078(B)

Table With Student Id Nd Name:

SELECT * FROM APP.STUDENT... x		
 Max. rows: 100 Fetched Rows: 10		
#	ID	NAME
1		1 sakshi
2		2 Aniket
3		3 Anuja
4		4 Sanjot
5		5 Pranay
6		6 shubham
7		7 sname
8		8 fname
9		10 praju
10		11 Manu

Practical No:6

Define a RESTful web service that accepts the details to be stored in a database and performs CRUD operation.

Step 1:create one table →employee→add Column→id,empname

Step2:create one project→new→Entity class→employee→package name→com.emp→data source→sample→finish

Step3:right click→inside class→add properties →column name—empname→on

Step4:right click on project→new→jsp pages from entity class→select→add entity class→ok

package=>1)com.emp.a

2)com.emp.b

3)c=>finish

Step5:right click on project →restful webServices from entity class→select→add Package name→com.emp.services→ finish

Step6:right click on project→clean and build→deploy→run

Code:

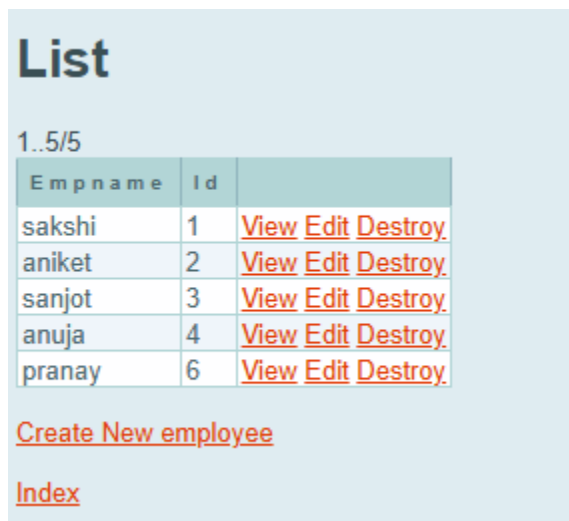
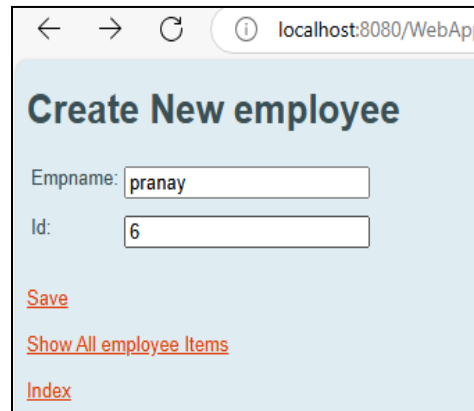
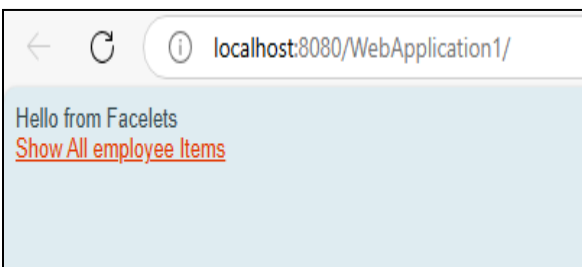
```
package com.emp;
import java.io.Serializable;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.xml.bind.annotation.XmlRootElement;
@Entity
@XmlRootElement
public class employee implements Serializable {
    private String empname;
    /**
     * Get the value of empname
     *
     * @return the value of empname
     */
    public String getEmpname() {
        return empname;
    }
    /**
     * Set the value of empname
```



```
*
* @param empname new value of empname
*/
public void setEmpname(String empname) {
    this.empname = empname;
}
private static final long serialVersionUID = 1L;
@Id
@GeneratedValue(strategy = GenerationType.AUTO)
private Long id;
public Long getId() {
    return id;
}
public void setId(Long id) {
    this.id = id;
}
@Override
public int hashCode() {
    int hash = 0;
    hash += (id != null ? id.hashCode() : 0);
    return hash;
}
@Override
public boolean equals(Object object) {
    // TODO: Warning - this method won't work in the case the id fields are not set
    if (!(object instanceof employee)) {
        return false;
    }
    employee other = (employee) object;
    if ((this.id == null && other.id != null) || (this.id != null && !this.id.equals(other.id))) {
        return false;
    }
    return true;
}
@Override
public String toString() {
    return "com.emp.employee[ id=" + id + " ]";
}
}
```

Name:Sakshi Parande
Roll N0:9078(B)

OutPut:



SELECT * FROM APP.EMPLOYEE... X		
Max. rows: 100 Fetched Rows: 5		
#	ID	EMPNAME
1		1 sakshi
2		2 aniket
3		3 sanjot
4		4 anuja
5		6 pranay

Practical No:5

Define a RESTful web service that returns the contents of a database in a JSON string. The contents should be displayed in a tabular format.

Step1:create one project

Step2:Right click on project→Restful Service from database→select →sample data→add one table—package name com.abc

Step3:double click on restfulServices→clean and build→deploy

Step4:Test WebServices→Restful Services →ok

Step5:Click on url→test→copy on data

Step6:Go back on Netbeans→remove mediaType.ApplicationXml→save→TestRestfulServices→Go To url

Step 7:Client Program→go to index .html→right code

Code :

```
<!DOCTYPE html>
```

```
<!--
```

To change this license header, choose License Headers in Project Properties.

To change this template file, choose Tools | Templates and open the template in the editor.

```
-->
```

```
<html>
```

```
  <head>
```

```
    <script>
```

```
      var request = new XMLHttpRequest();
```

```
      request.open("GET",
```

```
      "http://localhost:8080/9078sakshi/webresources/com.abc.customer", "true")
```

```
      request.onload= function() {
```

```
        var data = JSON.parse(this.response);
```

```
        var table = document.getElementById("myTable");
```

```
        for (var i=0; i<data.length;i++){
```

```
          var row = table .insertRow();
```

```
          var cell1= row.insertCell(0);
```

```
          var cell2= row.insertCell(1);
```

```
          cell1.innerHTML = data[i].addressline;
```

```
          cell2.innerHTML = data[i].city;
```

Name:Sakshi Parande
Roll N0:9078(B)

```
    }  
};  
request.send();  
  
</script>  
</head>  
<body>  
  <table id="myTable" border="1">  
    <tr>  
      <th>Address</th>  
      <th>City</th>  
    </tr>  
  </table>  
</body>  
</html>
```



Address	City
undefined	Fort Lauderdale
undefined	Miami
undefined	Houston
undefined	Alanta
undefined	San Mateo
undefined	San Jose
undefined	Santa Clara
undefined	Redwood City
undefined	Dearborn
undefined	Dearborn
undefined	Detroit
undefined	New York
undefined	New York

Practical No:7

Step1:open visual studio

Step2:Search —>select—>wcf services application—next

step3:Name of the project —>create project

Step4:go to the solution explorer

---->1)[services1.cs](#)==: 1) remove the all content below the operation contract

2)only operation contract is not remove

---->2)service1.svc=: 1) remove the below of the services1

2)select implement

3) remove throws exceptions

4) write return a+b;---->run

Step 5:select addition⇒put parameters ⇒click on invoke⇒ok

Step6:go to the [service1.cs](#)⇒run

⇒run [service1.cs](#)⇒copy url

Step7: again go to the visual studio⇒create new project⇒select [ASP.NET](#) web application ⇒language⇒c# ⇒next

Step8:Name of the project ⇒example ⇒clientApp⇒create

Step9:select empty⇒ create

Step10:go to the solution explorer⇒right click ⇒add⇒services references=>copy url⇒go⇒ok

Step11:add⇒new item⇒web from

Step12 : go to the toolbox⇒text box⇒button⇒label ⇒drag

Step13: Button ⇒select properties⇒double click on button ⇒ call the methods

Name:Sakshi Parande

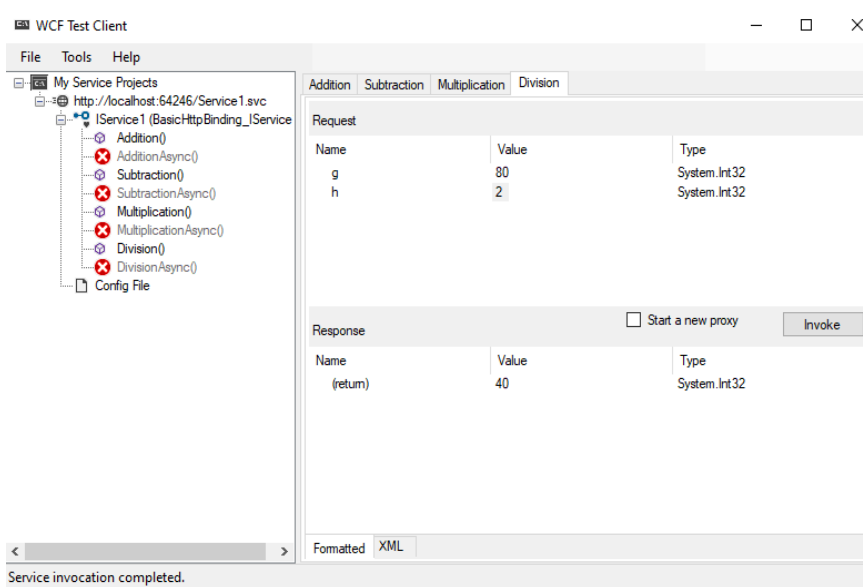
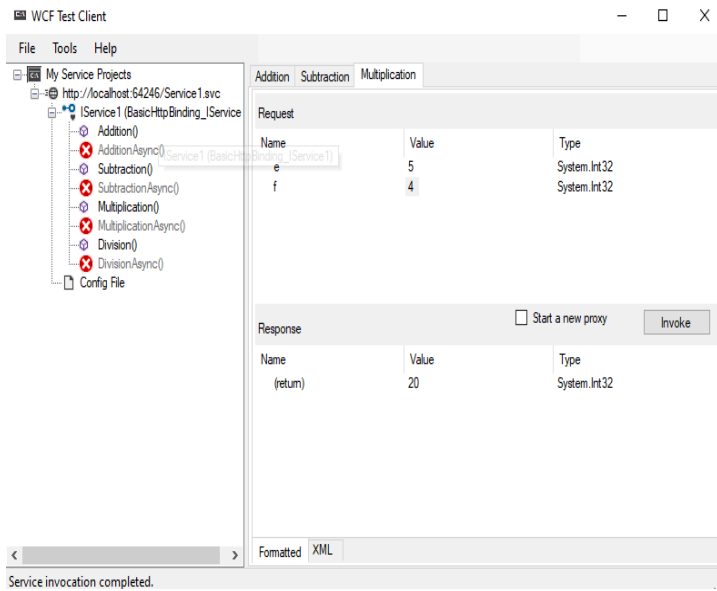
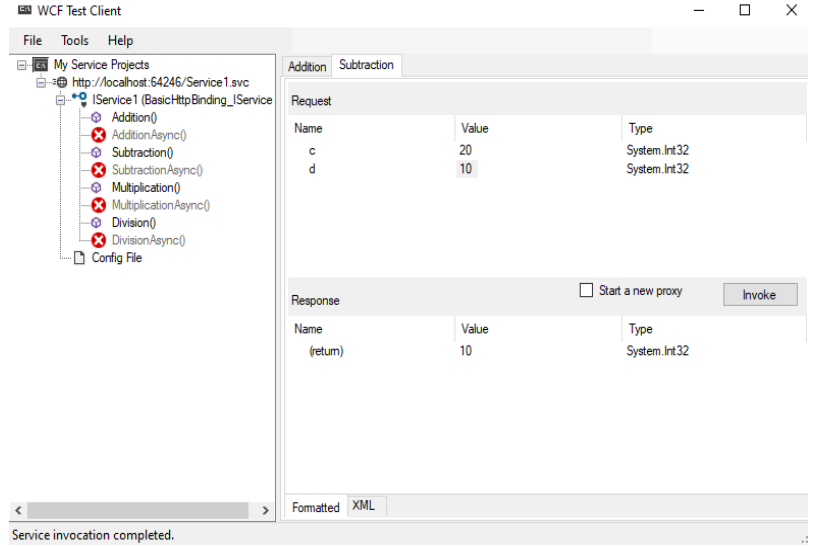
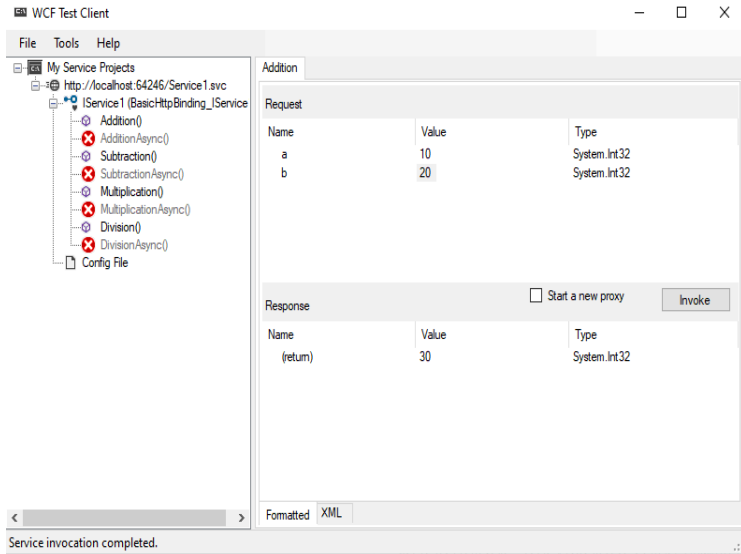
Roll N0:9078(B)

```
IService1.cs Service1.svc.cs sakshi: Overview What's New?
sakshi
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Runtime.Serialization;
5 using System.ServiceModel;
6 using System.ServiceModel.Web;
7 using System.Text;
8
9 namespace sakshi
10 {
11     // NOTE: You can use the "Rename" comm
12     [ServiceContract]
13     1 reference
14     public interface IService1
15     {
16         [OperationContract]
17         1 reference
18         int Addition(int a,int b);
19         [OperationContract]
20         1 reference
21         int Subtraction(int c,int d);
22         [OperationContract]
23         1 reference
24         int Multiplication(int e,int f);
25         [OperationContract]
26         1 reference
27         int Division(int g,int h);
28     }
29 }
```

```
Service1.svc.cs sakshi: Overview What's New?
sakshi.Service1
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Runtime.Serialization;
5 using System.ServiceModel;
6 using System.ServiceModel.Web;
7 using System.Text;
8
9 namespace sakshi
10 {
11     0 references
12     public class Service1 : IService1
13     {
14         1 reference
15         public int Addition(int a, int b)
16         {
17             return a + b;
18         }
19
20         1 reference
21         public int Subtraction(int c, int d)
22         {
23             return c - d;
24         }
25
26         1 reference
27         public int Multiplication(int e, int f)
28         {
29             return e * f;
30         }
31
32         1 reference
33         public int Division(int g, int h)
34         {
35             return g / h;
36         }
37     }
38 }
```

Name:Sakshi Parande

Roll N0:9078(B)



← → ↺ localhost:64246/Service1.svc

Service1 Service

You have created a service.

To test this service, you will need to create a client and use it to call the service. You can do this using the svcutil.exe tool from the command line with the following syntax:

```
svcutil.exe http://localhost:64246/Service1.svc?wsdl
```

You can also access the service description as a single file:

```
http://localhost:64246/Service1.svc?singleWsdl
```

This will generate a configuration file and a code file that contains the client class. Add the two files to your client application and use the generated client class to call the Service. For example:

```
C#  
  
class Test  
{  
    static void Main()  
    {  
        Service1Client client = new Service1Client();  
  
        // Use the 'client' variable to call operations on the service.  
  
        // Always close the client.  
        client.Close();  
    }  
}
```

WebForm1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Reflection.Emit;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace ClientApp3
{
    public partial class WebForm1 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {

        }

        protected void Button1_Click(object sender, EventArgs e)
        {
            int num1 = System.Convert.ToInt32(TextBox1.Text);
            int num2 = System.Convert.ToInt32(TextBox2.Text);

            ServiceReference1.Service1Client s1 = new ServiceReference1.Service1Client();
            Label1.Text = s1.Addition(num1, num2).ToString();

        }

        protected void Button2_Click(object sender, EventArgs e)
        {
            int num3= System.Convert.ToInt32(TextBox3.Text);
            int num4= System.Convert.ToInt32(TextBox4.Text);

            ServiceReference1.Service1Client s2 = new ServiceReference1.Service1Client();
            Label2.Text = s2.Subtraction(num3, num4).ToString();

        }

        protected void Button3_Click(object sender, EventArgs e)
        {
            int num5 = System.Convert.ToInt32(TextBox5.Text);
            int num6 = System.Convert.ToInt32(TextBox6.Text);

            ServiceReference1.Service1Client s3 = new ServiceReference1.Service1Client();
```

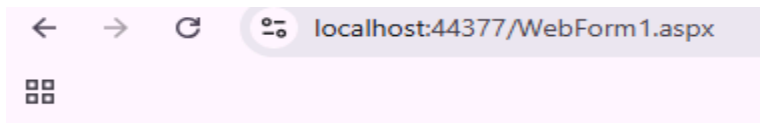


```
Label3.Text = s3.Multiplication(num5, num6).ToString();

}

protected void Button4_Click(object sender, EventArgs e)
{
    int num7 = System.Convert.ToInt32((string)TextBox4.Text);
    int num8 = System.Convert.ToInt32((string)TextBox5.Text);

    ServiceReference1.Service1Client s4 = new ServiceReference1.Service1Client();
    Label4.Text= s4.Division(num7, num8).ToString();
}
}
```



9078 Sakshi Parande

Addition

Result: 60

Subtraction

Result: 20

Multiplication

Result: 20

Division

Result: 2

Name:Sakshi Parande
Roll N0:9078(B)